

计算机组成原理

实验3 高速缓存器设计

2026 · 春

哈工大



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ

实验目的

- 掌握直接映射Cache的基本结构及工作原理
- 掌握直接映射Cache的HDL实现方法
- 认识Cache对计算机系统性能的影响



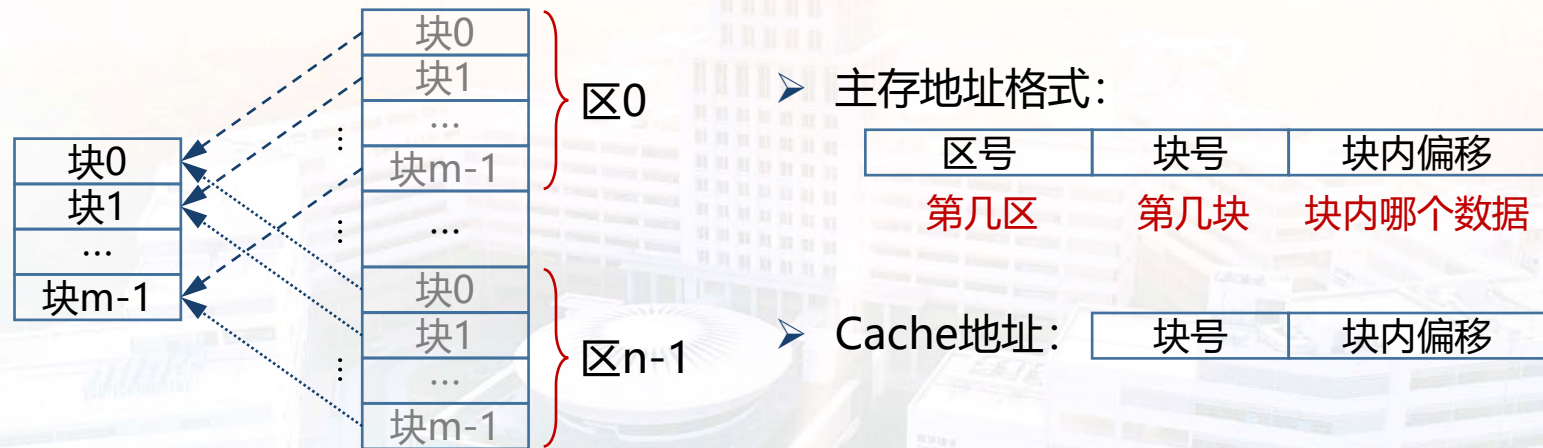
实验内容

- 在上一个实验的SoC工程中，设计实现直接映射的 ICache 和 DCache:
 1. 完成SoC工程中的 ICache 模块 (ICache.v)
 - 主存大小为**32KB**，采用Block Memory存储介质（带有随机访问延迟）
 - Cache大小为**1KB**，块大小为**16B**，采用Block Memory作为存储介质
 - ICache对CPU是只读的，故只需处理**读命中**、**读缺失**
 2. 完成SoC工程中的 DCache 模块 (DCache.v)，参数同上
 - DCache对CPU可读可写，故需处理**读命中**、**读缺失**、**写命中**、**写缺失**
 - DCache处理写请求时，采用**写直达**策略和**写不分配**策略
 - 处理**Uncached访问请求**，即绕过DCache直接向下游的总线发出访问请求
 3. 测试并**比较**无Cache和有Cache时的测试**程序运行时间**
 4. 基于Cache的结构和参数，优化GEMV算法（选做）

实验原理

1. 直接映射方式

- 主存被分为n个区，每个区的大小均与Cache相同



- 每个区中的第i块都能放在Cache中的第i块 —— 将区号作为tag来区分

实验原理

2. 地址分解

- 例：主存大小为256KB，Cache大小为4KB，数据块大小为128B

主存 **256KB** -> 主存地址**18bit**

Cache **4KB** -> Cache地址**12bit**

数据块 **128B** -> 块内偏移**7bit**

- 主存地址格式：



块号位宽 = **12bit** - **7bit** = **5bit**

区号位宽 = **18bit** - **12bit** = **6bit**

- Cache地址：



实验原理

3. Cache结构及访问原理

➤ Cache块由有效位、块标签、数据块组成

- 有效位 (v) : Cache块初始时无效,
缓存内存数据后有效
- 块标签 (tag) : 主存地址的区号
- 数据块 (data) : 多个连续的数据单元

v	tag	数据块			
⋮	⋮	⋮			

实验原理

3. Cache结构及访问原理

➤ Cache访问原理

① 地址划分

分解主存地址

② Cache块寻址

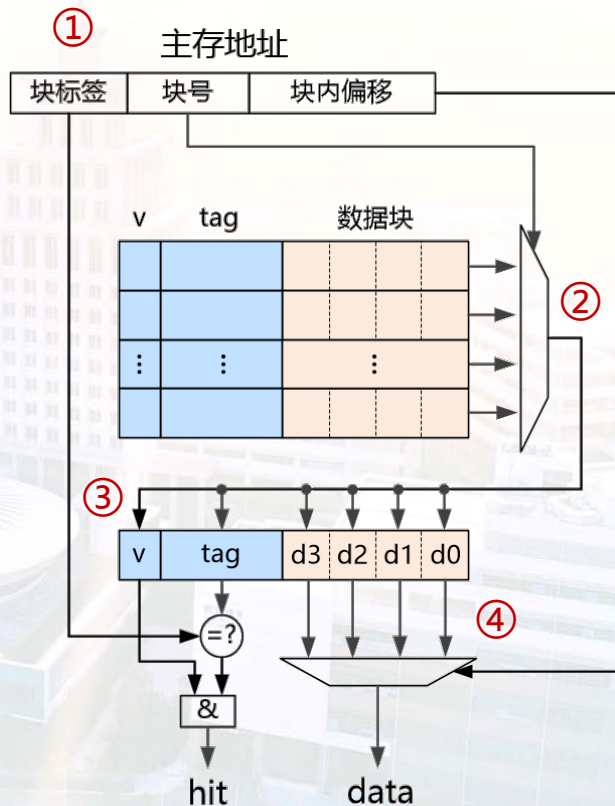
使用块号取出Cache块

③ 命中判断

$v \& (tag1 == tag2)$

④ 取出数据

使用块内偏移选出数据块



实验原理

3. Cache结构及访问原理

➤ Cache访问原理

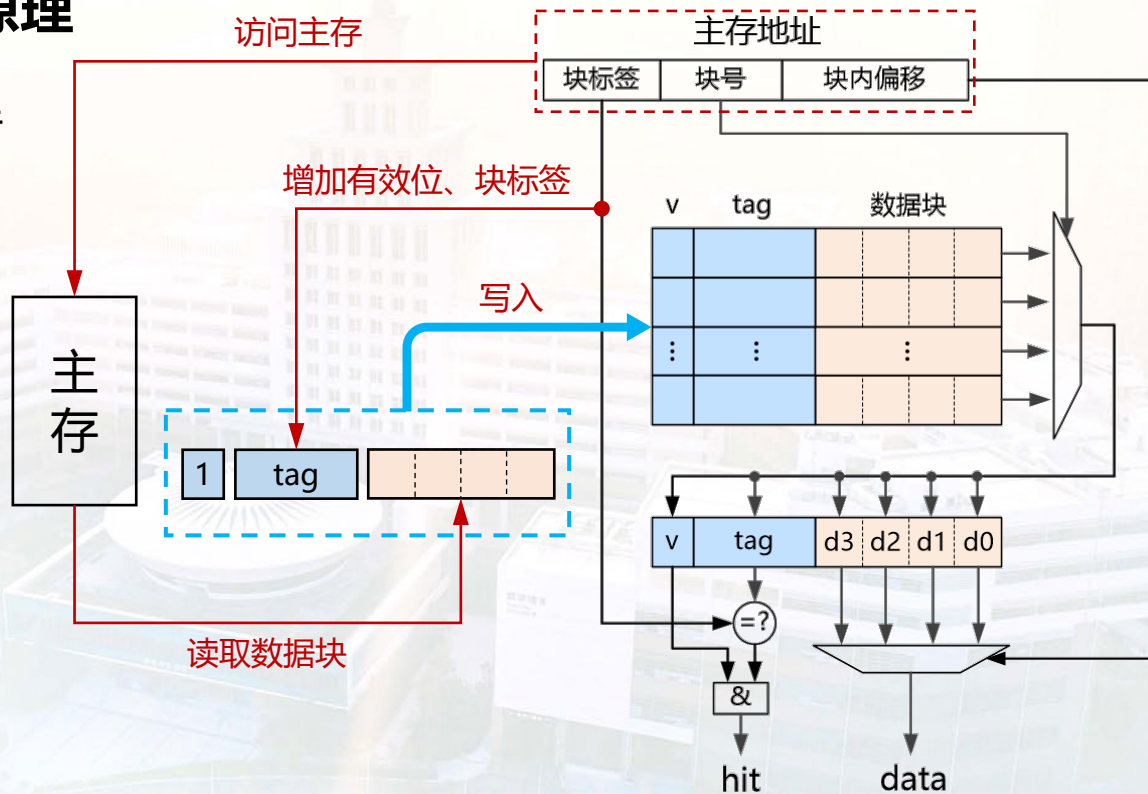
⑤ 缺失处理

读取内存

增加信息

写入Cache

判断命中



实验原理

4. Cache的读状态机

- **IDLE**: 就绪状态, 可以接收读请求
- **TAG_CHK**: 检查Cache是否命中
- **REFILL**: 读取主存, 为数据块增加有效位和块标签, 再将新的数据块写入Cache存储体



ICache.v

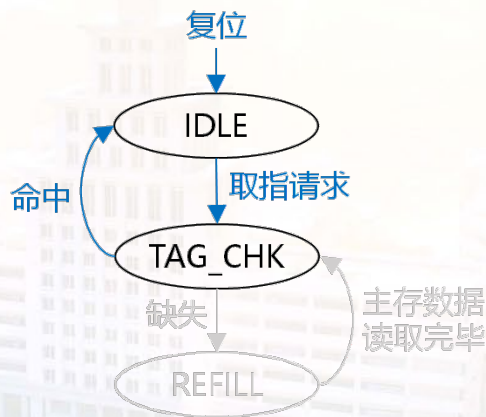
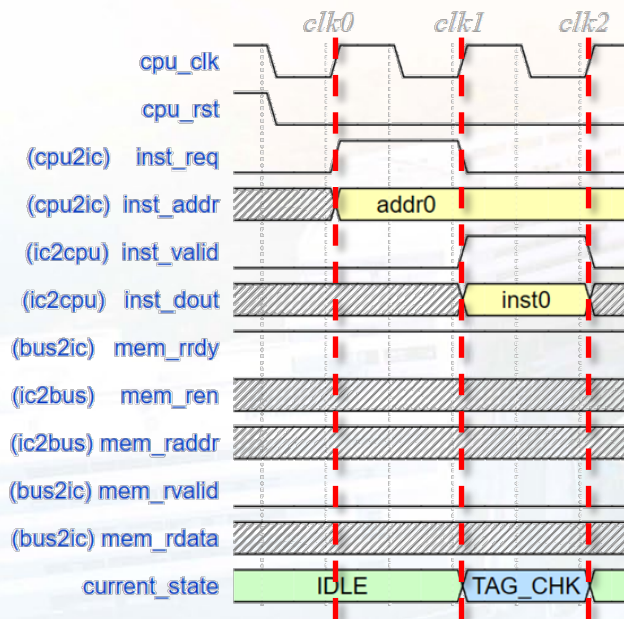
接口信号

名称	位宽	属性	功能描述
cpu_clk	1	输入	时钟信号
cpu_rst	1	输入	复位信号（高电平有效）
与CPU的接口			
inst_rreq	1	输入	CPU的取指请求
inst_raddr	32	输入	CPU的取指地址
inst_valid	1	输出	指令有效信号（命中信号）
inst_out	32	输出	返回给CPU的指令
与总线模块的接口			
dev_rrdy	1	输入	总线就绪信号（有效时ICache才能发出读请求）
cpu_ren	4	输出	读使能信号（对ICache，取值只能是4'h0或4'hF）
cpu_raddr	32	输出	读主存的地址信号
dev_rvalid	1	输入	总线返回的主存数据有效信号
dev_rdata	128	输入	总线返回的主存数据

实验原理

5. Cache的读时序

① 读命中



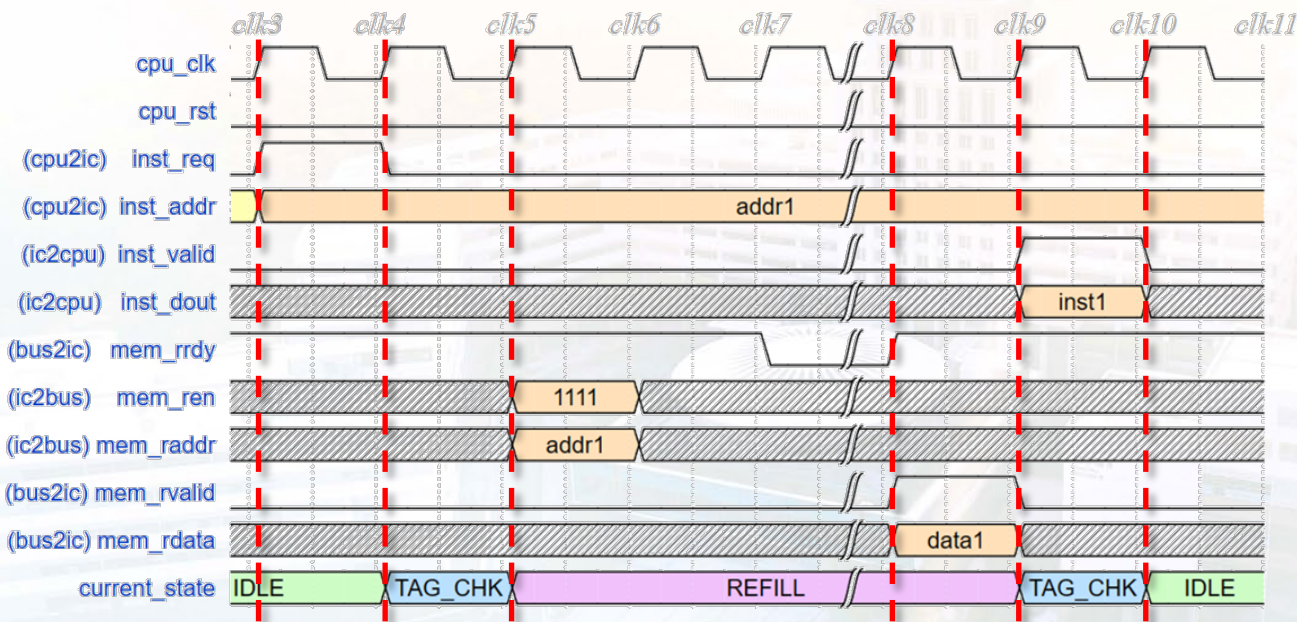
【clk0】CPU发出读请求

【clk1】Cache命中，输出命中信号和指令

实验原理

5. Cache的读时序

② 读缺失



【clk3】CPU发出读请求

【clk4】Cache缺失

【clk5】rrdy时读主存

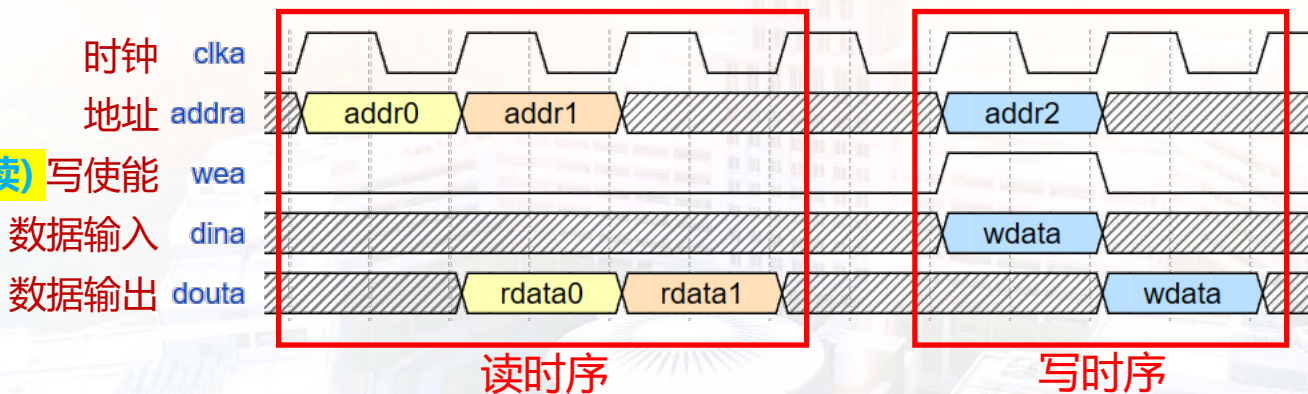
【clk8】主存数据返回

【clk9】命中并返回指令

实验原理

6. Cache存储体

- Cache采用Block Memory作为存储介质，已在模板代码中实例化好



- 读时序：当前时钟给出地址，下个时钟输出数据，可连续读取
- 写时序：当前时钟给出写地址、写数据、写使能wea，下个时钟数据被写入，且新写的数据被输出在douta端口上

DCache.v

读访存的接口信号

名称	位宽	属性	功能描述
cpu_clk	1	输入	时钟信号
cpu_rst	1	输入	复位信号（高电平有效）
与CPU的接口			
data_ren	4	输入	CPU的读数据请求（支持字节、半字、字）
data_addr	32	输入	CPU的 <u>读/写地址</u>
data_valid	1	输出	数据有效信号（ <u>命中信号</u> ）
data_rdata	32	输出	返回给CPU的数据
与总线模块的接口			
dev_rrdy	1	输入	总线就绪信号（ <u>有效时DCache才能发出读请求</u> ）
cpu_ren	4	输出	读使能信号（支持字节、半字、字）
cpu_raddr	32	输出	读主存的地址信号
dev_rvalid	1	输入	总线返回的主存数据有效信号
dev_rdata	128	输入	总线返回的主存数据

DCache.v

写访存的接口信号

名称	位宽	属性	功能描述
cpu_clk	1	输入	时钟信号
cpu_rst	1	输入	复位信号（高电平有效）
与CPU的接口			
data_wen	4	输入	CPU的写数据请求（支持字节、半字、字）
data_addr	32	输入	CPU的 <u>读/写地址</u>
data_wdata	32	输入	写数据信号
data_wresp	1	输出	返回给CPU的写相应（有效时表示写操作完成）
与总线模块的接口			
dev_wrdy	1	输入	总线就绪信号（ <u>有效时DCache才能发出写请求</u> ）
cpu_wen	4	输出	写使能信号（支持字节、半字、字）
cpu_waddr	32	输出	写主存的地址信号
dev_wdata	32	输出	写主存的写数据信号

实验原理

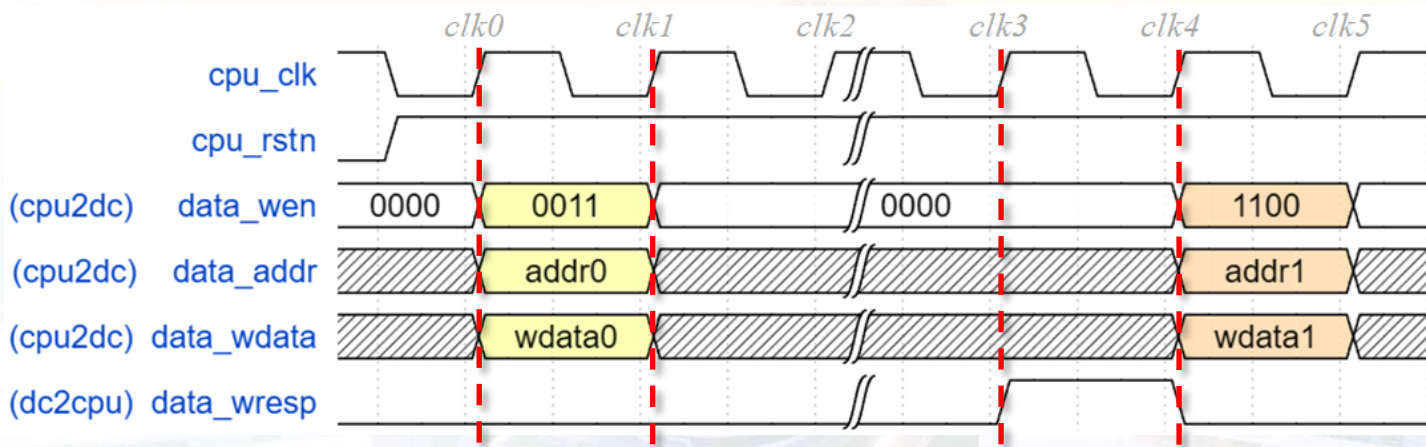
7. DCache写操作

- 要求采用写直达策略、写不分配策略
 - 写直达：每次写操作，不管写命中或写缺失，都要写主存
 - 写不分配：写缺失时，把数据写入主存，无须把数据块装入DCache
- 接收到CPU的写请求
 - └ 写命中：写Cache块、写主存
 - └ 写缺失：写主存
- 注意：一个Cache块包含多个32位字数据，写操作时需根据偏移量修改其中一个字数据

实验原理

8. DCache的写时序

➤ DCache与上游的CPU的接口时序：



【clk0~1】CPU发出写请求（写使能、写地址、写数据）

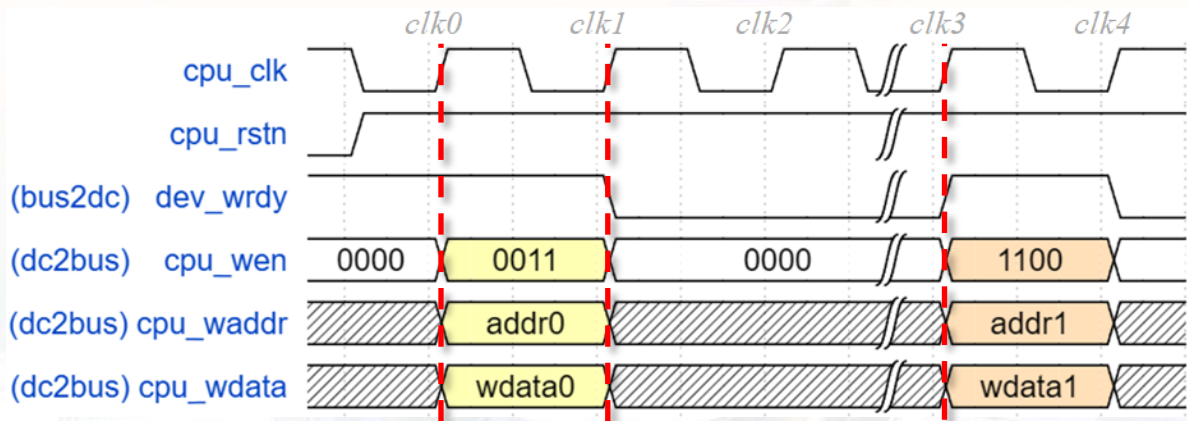
【clk3~4】Cache完成写操作时，拉高写响应信号data_wresp，告知CPU写操作已完成

注意：写请求、写响应均**只有效1个时钟周期**

实验原理

8. DCache的写时序

➤ DCache与下游的总线模块的接口时序



【clk0】总线就绪时，DCache向总线发出写请求（写使能、写地址、写数据）

【clk1】总线收到写请求后，拉低就绪信号dev_wrdy

【clk3】总线完成写操作时，重新拉高就绪信号dev_wrdy

注意：**DCache必须等到dev wrdy有效，才能发出写主存请求，且请求只有效1个时钟**

实验步骤

1. 理解直接映射Cache的结构及工作原理
2. 打开上一个实验的SoC工程，修改头文件defines.vh，启用ICache和内存延迟：

defines.vh

```
1 | // `define RANDOM_DELAY  
2 | // `define ENABLE_ICACHE
```



defines.vh

```
1 | `define RANDOM_DELAY  
2 | `define ENABLE_ICACHE
```

3. 按[实验指导书-实验步骤](#)，粘贴模板代码到ICache.v
4. 根据模板代码的提示，完成ICache.v，包括划分主存地址、编写命中判断的组合逻辑、设计状态机，etc.
5. 修改仿真顶层文件为soc_simu.v，运行功能仿真并完成调试
6. ICache通过测试后，再实现DCache（注意[Uncached读写请求](#)的处理）
7. 测试并对比有Cache和无Cache时，SoC运行测试程序的时间

附加题

- 在SoC工程中，启用ICache、DCache，测试并优化GEMV算法的性能
 - ① 增加GEMV的数据规模
 - ② 为减少仿真时间，仍使用带有随即延迟的Block Memory作为主存的存储介质
 - ③ 测试优化前的运行时间
 - ④ 结合硬件特性（Cache结构和参数）优化GEMV算法（可借助AI大模型）
 - ⑤ 测试优化后的运行时间
 - ⑥ 用同一种优化方法优化向量化前后的GEMV算法，对比优化效果

验收与提交

- **课上检查内容 (4分)**

- 启用ICache的SoC是否通过start.dump程序的仿真测试: 2分
- 启用DCache的SoC是否通过start.dump程序的仿真测试: 2分

- **提交内容 (8分)**

- Cache模块的.v源文件 (ICache.v、DCache.v) : 2分
- 实验报告 (按模板完成; 转成PDF提交) : 6分

- 将上述文件打包成.zip, 以 “学号_姓名.zip” 命名提交到作业系统

- ◆ DDL: **6月7日 23:59** (以作业系统为准; 迟交会扣迟交分)
- ◆ 注意: **如有雷同, 双方均0分!**



HITSZ 实验与创新实践教育中心
Education Center of Experiments and Innovations, HITSZ